

PACE 2026 Solver Description Uni-Bremen: Maximum Agreement Forest Problem

Florian Feegel  

University of Bremen, Germany

Leon Flaack 

University of Bremen, Germany

Enna Gerhard  

University of Bremen, Germany

Jurin Hoffmann 

University of Bremen, Germany

Philip Kail 

University of Bremen, Germany

André Kaufmann 

University of Bremen, Germany

Jonas Schramm  

University of Bremen, Germany

Sebastian Siebertz  

University of Bremen, Germany

Alexander Wachowski 

University of Bremen, Germany

Abstract

We describe our submission to the exact, heuristic, and lower-bound tracks of the PACE Challenge 2026 for the MAXIMUM AGREEMENT FOREST problem. The solver first applies a reduction rule as well as a clustering-based decomposition strategy to simplify the input instance and split it into smaller subproblems. Each resulting subproblem is solved independently by a branching algorithm, after which the partial solutions are merged to obtain the final solution.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Parameterized complexity and exact algorithms

Keywords and phrases PACE 2026, exact algorithms, heuristic algorithms, maximum agreement forest, rSPR, phylogenetics

Supplementary Material Public Repository: <https://github.com/pace26unibremen/PACE2026>

1 Introduction

This year's PACE challenge¹ features the MAXIMUM AGREEMENT FOREST problem, a classical NP-hard problem from phylogenetics [2]. Despite its name, the optimization task is usually formulated as a minimization problem: given several phylogenetic trees on the same set of labels, the goal is to cut them into as few common components as possible.

Let L be a finite set of leaf labels, and let $T_1, \dots, T_m$ be rooted phylogenetic trees with leaf-label set L . For a subset $X \subseteq L$, we write $T_i|X$ for the restriction of T_i to X , obtained by taking the minimal rooted subtree of T_i that spans the leaves in X and then suppressing

¹ <https://pacechallenge.org/2026/>

unlabeled vertices of indegree one and outdegree one. An agreement forest for $T_1, \dots, T_m$ is a forest $F = \{F_1, \dots, F_r\}$ whose component leaf sets $L(F_1), \dots, L(F_r)$ form a partition of L and satisfy the following two conditions. First, for every component F_j and every input tree T_i , the component F_j is isomorphic to the restricted tree $T_i|L(F_j)$, after the usual suppression of degree-two unlabeled vertices. Second, for each fixed input tree T_i , the subtrees of T_i spanning the different sets $L(F_1), \dots, L(F_r)$ are edge-disjoint.

2 Overview of the Solver

The solver described in the following is an exact solver for the MAF problem that comprises three distinct steps: preprocessing using a reduction rule and a clustering scheme, computing an upper bound via an approximation algorithm, and solving the preprocessed instance with a branching algorithm.

Starting with the preprocessing, described in detail in Section 3, the solver first exhaustively applies the subtree reduction rule to reduce the size of the instance. Afterwards, the instance is decomposed into several independent units using a clustering scheme. Each unit is subsequently solved independently, and the individual solutions are merged to reconstruct a solution for the original instance.

The units are solved using an approximation algorithm to obtain an upper bound on the size of an optimal agreement forest. This upper bound is then provided as input to the branching algorithm, allowing it to prune the search space and thereby reduce the overall running time. The approximation algorithms that have been implemented and the utilization of their results are broadly discussed in Section 4.

Finally, each unit is solved exactly using a branching algorithm, which applies reduction rules whenever possible and branches otherwise, following a depth-first search strategy. Section 5 elaborates on the branching algorithm and Section 6 on its correctness.

While the outlined algorithm was submitted to the exact track, it also serves as the basis for our submissions to the heuristic track and the lower-bound track, each incorporating minor modifications. These adaptations are described in Section 7 and Section 8, respectively.

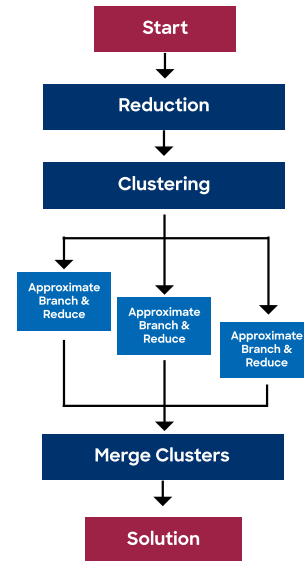


Figure 1 Basic algorithm outline.

3 Preprocessing with Reduction Rules and Clustering

The solver's preprocessing consists of two steps: the subtree reduction rule, which reduces the size of the instance, and a clustering scheme, which decomposes the instance into independent units that are solved separately and later merged.

3.1 Subtree reduction rule

The subtree reduction rule [3] identifies any maximal subtrees that occur identically across all input trees and contracts each such subtree into a single leaf. Since any shared subtree is necessarily preserved in an agreement forest, this contraction does not affect the size of an optimal solution while also reducing the number of leaves in the instance. Several variants of this rule have been proposed in the literature [1, 3, 4].

3.2 Clustering

After the subtree reduction has been applied exhaustively, the solver decomposes the remaining instance into smaller subinstances by cluster partitioning. For a rooted forest, a cluster is a set of leaf labels that occurs as the descendant set of a vertex. A cluster $C \subseteq X$ is common to the current instance if C is a cluster of every input forest. The induced subtrees on such a common cluster have the same leaf set in all forests, but their internal topologies may differ. Therefore, the disagreement inside C can be solved inside a separate MAF subinstance, while the rest of the instance only has to remember how this cluster is attached to the surrounding tree.

The implementation follows the cluster-partitioning approach used in fast MAF and MAAF algorithms [5]. The common clusters are used as split points. For each resulting cluster, the solver constructs a subinstance consisting of the corresponding induced subtrees in all forests. Nested common clusters are represented by artificial boundary leaves, so that each subinstance can be solved independently while the information needed to merge the solutions is retained. Once all subinstances have been solved, the solutions are combined by replacing the artificial boundary leaves with the corresponding cluster solutions and undoing the subtree reductions described above.

The common-cluster detection is a reimplementaion of the linear-time procedure described by Whidden and used in the RSPR software [8]. The additional bookkeeping needed to build the subinstances, store the artificial labels, and merge the partial solutions was implemented separately in the module CLUSTERSOLVER.

4 Approximation

The approximation routines in the solver are used only to seed later stages of the algorithm. They are not used to certify optimality in the exact track. We distinguish between two types of seeds. An *upper-bound seed* is the size of a feasible agreement forest, whereas a *lower-bound seed* must be a certified lower bound on the optimum.

4.1 Upper-bound seed

Before the branching procedure is invoked, the solver computes a feasible agreement forest for each current subinstance. If this forest has U components, then U is an upper bound on the optimum value of that subinstance. The branching algorithm uses this value as its initial incumbent: any search branch that cannot improve the incumbent can be pruned, and the incumbent also serves as a fall-back solution in the heuristic setting.

For subinstances consisting of two rooted binary trees, the upper-bound seed is computed with the linear-time 3-approximation algorithm of Whidden and Zeh [10]. The algorithm returns a valid agreement forest whose size is within a factor three of the optimum for the two-tree case.

For instances with more than two input forests, the Whidden–Zeh algorithm does not apply directly. In this case the solver falls back to a greedy cutting procedure. The greedy procedure repeatedly resolves local disagreements by cutting edges until the resulting forests agree. It has no proven approximation ratio, but it always returns a feasible agreement forest. This is sufficient because the result only provides a starting solution for the branching solver and a fall-back solution, and is never used as a guaranteed-quality answer.

4.2 Lower-bound seeds

The implementation can also be supplied with a seed pair (L, U) , where U is the size of a feasible agreement forest and L is a certified lower bound on the optimum. The two values have different roles: U initializes or improves the best solution found so far, while L is used only when a certificate is available. In particular, the lower bound is not obtained from the Whidden–Zeh upper-bound routine.

For the lower-bound track, the solver computes certified lower bounds from feasible dual solutions in the duality-based approximation framework for MAF on two rooted binary trees [7, 6]. The inexpensive bound associated with the 3-approximation is computed first, and the stronger bound associated with the 2-approximation may later improve the certificate. Since these values are relevant only for the early stopping criterion of the lower-bound submission, their construction and use are described in Section 8.

5 Branching

The solver uses a branching technique that tries to reduce the size of the search space. It applies cuts splitting the trees $T_1, \dots, T_k$ into forests $F_1, \dots, F_k$ until $F_1 = F_2 = \dots = F_k$. For the smallest number of cuts, the minimum agreement forest F has been found in this way. Several rules from the literature are employed that both reduce the size of the instance after branching and determine for which cuts resemble a single branching decision.

Here, branching is facilitated by recursively checking the following branching rules in order and choosing the first applicable. Apart from the first rule, all rules are based on the selection of a pair of terminals a, c in the first forest F_1 , which is subsequently compared with all other forests.

5.1 Single Vertex Tree Propagation

Trivially, if one of the forests $F_1, \dots, F_k$ contains a single vertex tree, any edges connected to leaves of the same label need to be cut in all other forests.

5.2 AC (Separate components)

This rule seen in Whidden [8, p.107, "Separate components (AC)"] is applicable if for all other forests:

1. the leaves a and c are siblings, or
2. they belong to separate connected components.

In this case, 2-branching occurs, where either a or c is cut. In the second branch where c is cut, edge protection is applied to the edge connected to a .

5.3 EqualPair

This reduction rule is applicable in the trivial case where a and c are siblings in all forests. In fact, it is a simplified version of the subtree reduction described in 3.1. Here, a and c get contracted into one representative terminal.

5.4 B (One pendant subtree)

This rule adapted from [8, p.107, "One pendant subtree (B)"] is applicable if one of two cases is true in all other forests for some leaves b_1, b_2 :

1. the leaves a and c are siblings, or
2. either a or c forms a sibling pair with b_i , and the other is the uncle of the pair.

In this case, the terminals b_i are cut in all forests.

5.5 RB (Reverse case B)

This rule in [8, p.109, "Reverse Case B (RB)"] is applicable for a pair of terminals a, c and their terminal uncle x in one forest if one of two cases is true in all other forests:

1. the leaves a and c are siblings and x their uncle, or
2. either a or c forms a sibling pair with x .

In this case, the leaf that does not form the sibling pair is cut.

5.6 2B (Two pendant subtrees and a common uncle)

This rule adapted from [8, p.109, "Two pendant subtrees and a common uncle (2B)"] is applicable for a pair of terminals a, c and their terminal uncle x in one forest if one of three cases is true in all other forests for some leaves b_1, b_2 :

1. the leaves a and c are siblings and x their uncle, or
2. either a or c forms a sibling pair with b_i with the remaining leaf from a, b as their uncle and x as their great uncle.
3. either a or c forms a sibling pair with b_i with the other b_j as their uncle, the remaining leaf from a, b their great uncle and x as their great great uncle.

In this case, the terminals b_i are cut in all forests.

5.7 ABC (Multiple pendant subtrees)

This rule in [8, p.109, "Multiple pendant subtrees (A3BC)"] is applicable if the sibling pair a, c is part of the same connected component in all forests.

In this case, 3-branching occurs, where either a is cut, all incident edges of the path between a and c are cut or c is cut. In the third branch where c is cut, edge protection is applied to the edge connected to a .

6 Sketch of correctness

The correctness of the rules for two trees using the branching algorithm was proven in their respective sources [3, 5, 6, 8, 9]. Expanding *Equal Pair*, *AC*, *ABC*, *2B* and *RB* to multiple trees is trivial. The *B*-Rule can be applied with two different b 's across the trees since not applying the rule and separating the underlying pair ac would result in at least the same cost later on as applying *B* on different b 's.

199 **7** Heuristic track

200 The heuristic track contains two-tree instances whose size makes a complete exact search
 201 unlikely within the time limit. We therefore run the pipeline from Sections 3–5 as an anytime
 202 algorithm. However, after preprocessing and cluster decomposition, each cluster is initialized
 203 with a feasible agreement forest obtained from the approximation routine. This gives the
 204 branching solver a starting solution before the first branch is explored. The solver then
 205 performs a depth-first search and continuously replaces the current best solution whenever it
 206 finds a smaller agreement forest.

207 To prevent a single difficult cluster from consuming the entire time budget, the solver
 208 distributes the remaining time over the independent clusters produced by Section 3.2. The
 209 share assigned to a cluster is proportional to a difficulty estimate that combines the number
 210 of leaves in the cluster with the size of its initial agreement forest.

211 Since a feasible solution is available from the beginning, the search can be interrupted at
 212 any point and still return a valid agreement forest. In particular, when the process receives a
 213 **SIGTERM** signal at the end of the time limit, the signal handler writes out the best solution
 214 found so far before the program terminates.

215 **8** Lower-bound track

216 The lower-bound track also consists of instances with two input trees, but unlike the heuristic
 217 track it does require the solver to prove optimality within an instance-specific threshold.
 218 Thus, an interrupted search is not necessarily valid: the current incumbent may be feasible,
 219 but it may fail the required approximation guarantee. For this reason, the solver does not
 220 simply terminate on **SIGTERM** and return the best solution found so far.

221 We also disable the clustering decomposition for this track. Instead, the complete instance
 222 is passed through the reduction and branching stages. The branching solver is initialized
 223 with the approximation described in Section 4, which provides an initial feasible incumbent.
 224 It then continues until either optimality within the threshold is proven.

225 The threshold is based on a lower bound L on the unknown optimum, that is, on a value
 226 satisfying $L \leq k^*$. This lower bound is computed independently of the incumbent solution.
 227 We use the LP-duality analysis of Olver et al. [6]: their approximation algorithms construct
 228 a feasible solution to the dual of a linear-programming relaxation of MAF. By weak duality,
 229 the value of any feasible dual solution is a lower bound on the optimum value of the primal
 230 relaxation, and hence on the optimum number of cuts. If this dual value is denoted by D ,
 231 then $D \leq k^* - 1$, because an agreement forest of size k^* is obtained by making $k^* - 1$ cuts.
 232 Therefore $L := D + 1$ is a valid lower bound on the optimum agreement-forest size.

233 The solver uses two such lower bounds. First, it computes the inexpensive bound obtained
 234 from the dual solution associated with the 3-approximation. This gives a certified bound
 235 quickly and may already allow an early exit. In parallel, it computes the stronger but more
 236 expensive bound associated with the 2-approximation. Once this second bound becomes
 237 available, the solver updates L to the larger of the available certified lower bounds.

238 Given a valid lower bound L , we define the certified acceptance threshold $\theta := \lfloor aL \rfloor + b$.
 239 Since $L \leq k^*$ and the map $x \mapsto \lfloor ax \rfloor$ is monotone for $a \geq 0$, we have $\theta = \lfloor aL \rfloor + b \leq \lfloor ak^* \rfloor + b$.
 240 Consequently, if the current incumbent has size at most θ , then it already satisfies the
 241 lower-bound track requirement. At that point the solver may safely terminate, even if
 242 optimality has not been proven.

Correctness

The solver never returns an infeasible agreement forest, and it never returns a solution whose size exceeds the accepted bound. Feasibility follows from the way solutions are recorded: the branching search only stores a solution once the current state represents an agreement forest, and the initial incumbent produced by the approximation algorithm is an agreement forest as well. The reductions and branching rules used during the search are correctness-preserving, as argued in Section 6.

It remains to justify the size guarantee. If the solver terminates after proving optimality, then the returned solution has size k^* and is therefore accepted. Otherwise, the solver terminates by the certified early-exit condition. In this case, the returned incumbent has size $s \leq \theta$. Since $\theta \leq \lfloor ak^* \rfloor + b$, it follows that $s \leq \lfloor ak^* \rfloor + b$. Thus the returned solution satisfies the acceptance criterion of the lower-bound track.

The only assumption used by the early-exit argument is that $L \leq k^*$. This holds because $L - 1$ is the objective value of a feasible dual solution for an LP relaxation of MAF. By weak duality, this value is a lower bound on the minimum number of cuts, which is $k^* - 1$ for an optimum agreement forest of size k^* [6]. Hence $L \leq k^*$.

References

- 1 Magnus Bordewich and Charles Semple. On the computational complexity of the rooted subtree prune and regraft distance. *Annals of combinatorics*, 8(4):409–423, 2005.
- 2 Jotun Hein, Tao Jiang, Lusheng Wang, and Kaizhong Zhang. On the complexity of comparing evolutionary trees. *Discrete Applied Mathematics*, 71(1-3):153–169, 1996.
- 3 Steven Kelk, Simone Linz, and Ruben Meuwese. Cyclic generators and an improved linear kernel for the rooted subtree prune and regraft distance. *Information Processing Letters*, 180:106336, 2023.
- 4 Steven Kelk, Ruben Meuwese, and Leo van Iersel. A kernel for the maximum agreement forest problem on multiple binary phylogenetic trees. *arXiv preprint arXiv:2602.22281*, 2026.
- 5 Zhijiang Li and Norbert Zeh. Computing Maximum Agreement Forests without Cluster Partitioning is Folly. In Kirk Pruhs and Christian Sohler, editors, *25th Annual European Symposium on Algorithms (ESA 2017)*, volume 87 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 56:1–56:14, Dagstuhl, Germany, 2017. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2017.56>, doi:10.4230/LIPIcs.ESA.2017.56.
- 6 Neil Olver, Frans Schalekamp, Suzanne van der Ster, Leen Stougie, and Anke van Zuylen. A duality based 2-approximation algorithm for maximum agreement forest. *Mathematical Programming*, 198:811–853, 2023. doi:10.1007/s10107-022-01790-y.
- 7 Frans Schalekamp, Anke van Zuylen, and Suzanne van der Ster. A duality based 2-approximation algorithm for maximum agreement forest. In *43rd International Colloquium on Automata, Languages, and Programming (ICALP 2016)*, volume 55 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 70:1–70:14. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2016. doi:10.4230/LIPIcs.ICALP.2016.70.
- 8 Chris Whidden. *Efficient Computation and Application of Maximum Agreement Forests*. PhD thesis, Dalhousie University, 2013.
- 9 Chris Whidden, Robert G Beiko, and Norbert Zeh. Fixed-parameter algorithms for maximum agreement forests. *SIAM Journal on Computing*, 42(4):1431–1466, 2013.
- 10 Chris Whidden and Norbert Zeh. A unifying view on approximation and FPT of agreement forests. In *Algorithms in Bioinformatics (WABI 2009)*, volume 5724 of *Lecture Notes in Computer Science*, pages 390–402. Springer, 2009. doi:10.1007/978-3-642-04241-6_32.